

Entrega 4

Hugo Jaime Valdés A01384039

2025-10-24

```
# Proyecto Métodos Multivariados – Clustering 2024 (Ward.D2)
# Requisitos:
#   - Tres archivos (uno por estación) con mediciones HORARIAS.
#   - Debe existir una columna de fecha/tiempo (p. ej. "fecha",
#     "datetime", "time", etc.).
#   - Columnas de contaminantes: CO, NO, NO2, O3, PM10, PM2.5, SO2, etc.
#   - El script unifica nombres (PM2.5 -> pm2_5) y detecta
#     automáticamente la columna fecha.
#
# Salidas:
#   1) Dendrograma (Ward.D2) sobre promedios MENSUALES 2024 por estación.
#   2) k sugerido (2..6) por anchura de silueta.
#   3) Tabla con clúster por (Estación, Mes).
#   4) Gráfico de medias por clúster (variables escaladas).
```

```
# 0) Paquetes y semilla
```

```
suppressPackageStartupMessages({
  library(readxl)
  library(dplyr)
  library(tidyr)
  library(lubridate)
  library(janitor)
  library(ggplot2)
  library(cluster)
  library(factoextra) # para silhouette y dendrograma bonito
})
```

```
set.seed(1234)
```

```
# 1) Rutas de archivos
```

```
FILE_CAD <- "cadereyta_SE3_2024.xlsx" # ej: "cadereyta_SE.xlsx" /
"cadereyta_2023_2025.xlsx"
FILE_JUA <- "juarez_SE2_2024.xlsx"    # ej: "juarez_SE.xlsx"
FILE_APO <- "apodaca_NE2_2024.xlsx"   # ej: "apodaca_SE.xlsx"
```

```
# 2) Utilidades
```

```
parse_datetime_smart <- function(df){
  cand <- names(df)
```

```

target_idx <-
grep("^(fecha|fechahora|datetime|date_time|time|timestamp|hora)$",
      tolower(cand))
if (length(target_idx) == 0) {
  target_idx <- grep("(fecha|date|time|hora|timestamp)", tolower(cand))
}
if (length(target_idx) == 0) {
  stop(" No se encontró columna de fecha/tiempo. Renombra o indica la
columna correcta.")
}
col <- cand[target_idx[1]]
if (inherits(df[[col]], "POSIXct") || inherits(df[[col]], "Date")) {
  dt <- as.POSIXct(df[[col]])
} else {
  dt_try <- suppressWarnings(parse_date_time(
    df[[col]],
    orders = c("Ymd HMS", "Ymd HM", "Ymd H", "Ymd",
               "dmY HMS", "dmY HM", "dmY",
               "Y-m-d H:M:S", "Y-m-d H:M", "Y-m-d",
               "d/m/Y H:M:S", "d/m/Y H:M", "d/m/Y",
               "m/d/Y H:M:S", "m/d/Y H:M", "m/d/Y"),
    tz = "UTC"
  ))
  if (all(is.na(dt_try))) stop(" No se pudo parsear la columna de
fecha/tiempo.")
  dt <- as.POSIXct(dt_try)
}
df$fecha_hora <- dt
df
}

read_station <- function(path, estacion_name){
  df <- suppressMessages(read_excel(path))
  df <- janitor::clean_names(df)
  df <- parse_datetime_smart(df)
  df$estacion <- estacion_name
  df
}

# Mapeo manual de meses (evita bugs de Locale)
mes_labels_es <-
c("Ene", "Feb", "Mar", "Abr", "May", "Jun", "Jul", "Ago", "Sep", "Oct", "Nov", "Dic"
)

# Posibles variables (se filtran las que existan)
possible_vars <-
c("co", "no", "no2", "o3", "pm10", "pm2_5", "so2_ppm", "prs", "rh", "sr", "tout", "w
sr", "wdr", "")

```

3) Lectura y unión de bases

```
cad <- read_station(FILE_CAD, "SE3")
jua <- read_station(FILE_JUA, "SE2")
apo <- read_station(FILE_APO, "NE2")
```

```
raw_all <- bind_rows(cad, jua, apo)
```

Año 2024 y mes como factor en español (sin locale)

```
raw_all <- raw_all %>%
  mutate(
    anio = year(fecha_hora),
    mes_num = month(fecha_hora),
    mes = factor(mes_num, levels = 1:12, labels = mes_labels_es)
  ) %>%
  filter(anio == 2024)
```

```
vars_exist <- intersect(possible_vars, names(raw_all))
if (length(vars_exist) == 0) {
  stop(" Ninguna variable candidata está presente. Revisa nombres de
columnas.")
}
```

4) Agregación MENSUAL (promedio por estación y mes)

```
monthly <- raw_all %>%
  group_by(estacion, anio, mes) %>%
  summarise(across(all_of(vars_exist), ~mean(.x, na.rm = TRUE)), .groups
= "drop")
```

```
monthly_unscaled <- monthly
```

5) Escalado de variables

```
X <- monthly %>%
  select(all_of(vars_exist)) %>%
  mutate(across(everything(), ~replace_na(.x, median(.x, na.rm = TRUE))))
%>%
  as.data.frame()
```

```
na_cols <- names(X)[sapply(X, function(col) all(is.na(col)))]
if (length(na_cols) > 0) {
  message("Columnas eliminadas por estar completamente vacías en 2024: ",
    paste(na_cols, collapse = ", "))
  X <- X %>% select(-all_of(na_cols))
  vars_exist <- setdiff(vars_exist, na_cols)
}
```

```
X_scaled <- scale(X)
```

```
row_ids <- monthly %>% transmute(id = paste(estacion, as.character(mes),
sep = "_"))
```

6) Clustering jerárquico (Ward.D2) y dendrograma [FIX Labels]

Etiquetas "Estación_Mes" como rownames para que se propaguen a dist/hclust

```
nice_labels <- paste(monthly$estacion, as.character(monthly$mes), sep =
"_")
```

```
stopifnot(length(nice_labels) == nrow(X_scaled)) # QC
```

```
rownames(X_scaled) <- nice_labels
```

Distancia y clustering

```
d <- dist(X_scaled, method = "euclidean")
```

```
hc <- hclust(d, method = "ward.D2")
```

Dendrograma (SIN 'Labels = ...')

```
fviz_dend(
  hc, k = NULL, cex = 0.9,
  main = "Dendrograma - Promedio mensual 2024 (Ward.D2)",
  xlab = "Estación_Mes", ylab = "Altura (inercia)"
)
```

```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
```

```
## i Please use tidy evaluation idioms with `aes()`.
```

```
## i See also `vignette("ggplot2-in-packages")` for more information.
```

```
## i The deprecated feature was likely used in the factoextra package.
```

```
## Please report the issue at
```

```
<https://github.com/kassambara/factoextra/issues>.
```

```
## This warning is displayed once every 8 hours.
```

```
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning
was
```

```
## generated.
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2
3.4.0.
```

```
## i Please use `linewidth` instead.
```

```
## i The deprecated feature was likely used in the factoextra package.
```

```
## Please report the issue at
```

```
<https://github.com/kassambara/factoextra/issues>.
```

```
## This warning is displayed once every 8 hours.
```

```
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning
was
```

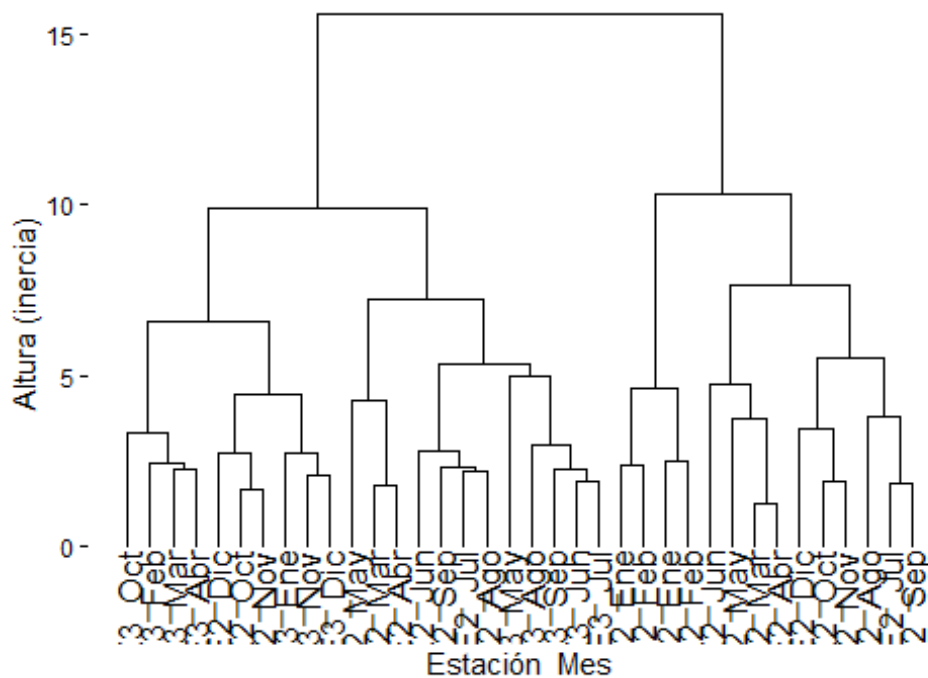
```
## generated.
```

```
## Warning: The `<scale>` argument of `guides()` cannot be `FALSE`. Use
"none" instead as
```

```
## of ggplot2 3.3.4.
```

```
## i The deprecated feature was likely used in the factoextra package.
## Please report the issue at
<https://github.com/kassambara/factoextra/issues>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning
was
## generated.
```

Dendrograma — Promedio mensual 2024 (Ward.D2)



```
# 7) Sugerencia de k por silueta (2..6)
k_range <- 2:6
sil_vals <- sapply(k_range, function(k){
  cutk <- cutree(hc, k = k)
  sil <- silhouette(cutk, d)
  mean(sil[, "sil_width"])
})
k_best <- k_range[which.max(sil_vals)]
cat("\n>> Anchura media de silueta por k (2..6):\n")

##
## >> Anchura media de silueta por k (2..6):

print(setNames(round(sil_vals, 3), paste0("k=", k_range)))

## k=2 k=3 k=4 k=5 k=6
## 0.252 0.243 0.205 0.214 0.223

cat(">> k sugerido por silueta:", k_best, "\n\n")
```

```
## >> k sugerido por silueta: 2

k <- k_best
grp <- cutree(hc, k = k)

# 8) Tabla de clúster por Estación y Mes
clusters_tbl <- monthly %>%
  select(estacion, mes, all_of(vars_exist)) %>%
  mutate(cluster = factor(grp, labels = paste0("G", 1:k))) %>%
  arrange(estacion, mes)

cat("=== Asignación de clúster (mensual, 2024) ===\n")

## === Asignación de clúster (mensual, 2024) ===

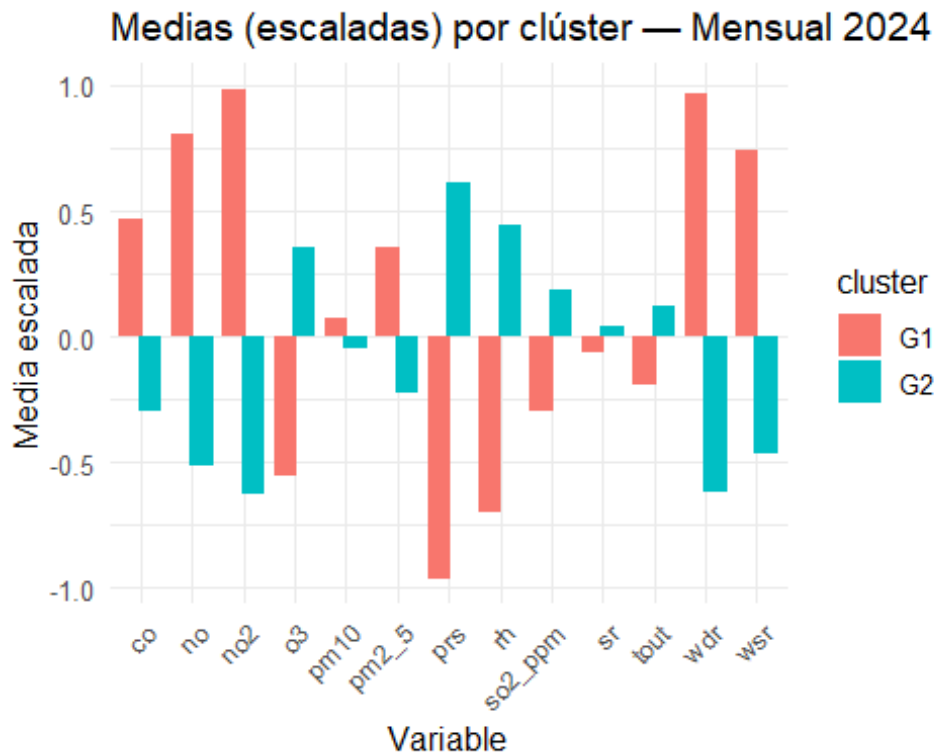
print(clusters_tbl %>% select(estacion, mes, cluster))

## # A tibble: 36 × 3
##   estacion mes   cluster
##   <chr>    <fct> <fct>
## 1 NE2     Ene    G1
## 2 NE2     Feb    G1
## 3 NE2     Mar    G1
## 4 NE2     Abr    G1
## 5 NE2     May    G1
## 6 NE2     Jun    G1
## 7 NE2     Jul    G1
## 8 NE2     Ago    G1
## 9 NE2     Sep    G1
## 10 NE2    Oct    G1
## # i 26 more rows

# 9) Medias (escaladas) por clúster
plot_df <- as.data.frame(X_scaled)
plot_df$cluster <- factor(grp, labels = paste0("G", 1:k))
plot_df$EstacionMes <- row_ids$id

plot_long <- plot_df %>%
  pivot_longer(cols = all_of(vars_exist), names_to = "variable",
  values_to = "valor")

ggplot(plot_long, aes(x = variable, y = valor, fill = cluster)) +
  stat_summary(fun = mean, geom = "col", position = position_dodge(width
= 0.75)) +
  labs(title = "Medias (escaladas) por clúster – Mensual 2024",
  x = "Variable", y = "Media escalada") +
  theme_minimal(base_size = 12) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



```
# 10) (Opcional) Resumen DIARIO con clústeres
#      (proyecta días al mismo espacio escalado)
daily <- raw_all %>%
  mutate(fecha = as.Date(fecha_hora)) %>%
  group_by(estacion, fecha) %>%
  summarise(across(all_of(vars_exist), ~mean(.x, na.rm = TRUE)), .groups
= "drop")

dailyX <- daily %>%
  select(all_of(c("estacion", "fecha", vars_exist))) %>%
  mutate(across(all_of(vars_exist), ~replace_na(.x, median(.x, na.rm =
TRUE))))

# Variables comunes entre mensual (X_scaled) y diario
common_vars <- intersect(colnames(X), colnames(dailyX))

# Usa los mismos parámetros de escalado del mensual
center_vec <- attr(X_scaled, "scaled:center")[common_vars]
scale_vec <- attr(X_scaled, "scaled:scale")[common_vars]

daily_mat <- as.matrix(dailyX[common_vars])
daily_scaled <- scale(daily_mat, center = center_vec, scale = scale_vec)

# Centroides mensuales por cluster (en espacio escalado)
centroides <- plot_df %>%
  group_by(cluster) %>%
```

```

summarise(across(all_of(common_vars), mean), .groups = "drop")

assign_to_centroid <- function(xrow, centers){
  dists <- apply(centers, 1, function(crow) sqrt(sum((xrow - crow)^2)))
  which.min(dists)
}

centers_mat <- as.matrix(centroides[, common_vars])
daily_assign_idx <- apply(daily_scaled, 1, assign_to_centroid, centers =
centers_mat)
daily_clusters <- factor(daily_assign_idx, labels =
levels(plot_df$cluster))

daily_out <- dailyX %>%
  mutate(cluster = daily_clusters) %>%
  select(estacion, fecha, cluster)

cat("\n=== Resumen diario: conteo de días por clúster y estación (2024)
===\n")

##
## === Resumen diario: conteo de días por clúster y estación (2024) ===

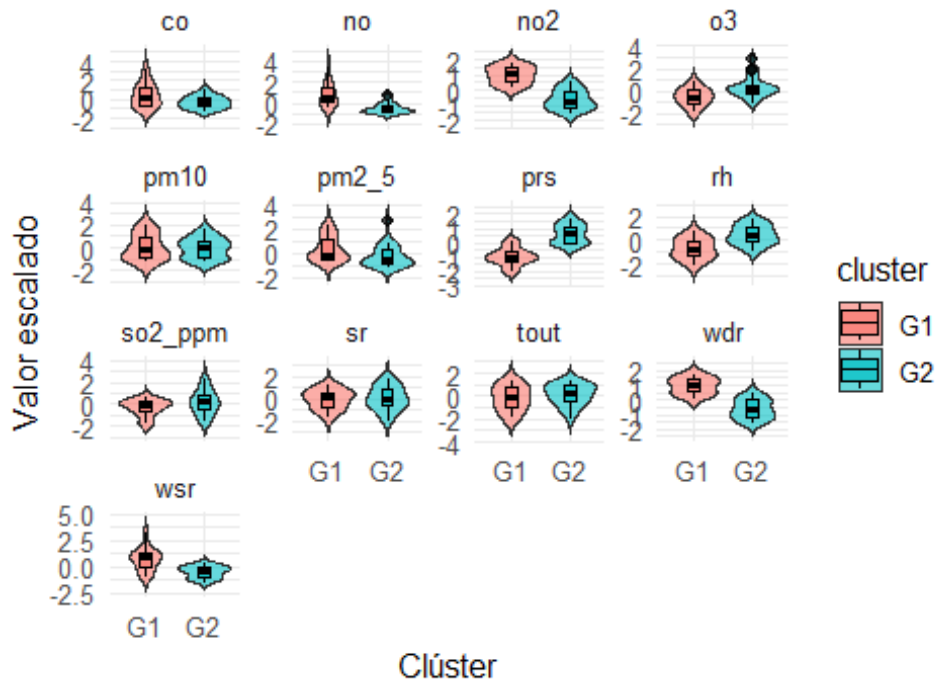
print(daily_out %>% count(estacion, cluster) %>% arrange(estacion,
cluster))

## # A tibble: 6 × 3
##   estacion cluster      n
##   <chr>    <fct>   <int>
## 1 NE2      G1       320
## 2 NE2      G2        46
## 3 SE2      G1        75
## 4 SE2      G2       291
## 5 SE3      G1        40
## 6 SE3      G2       326

ggplot(plot_long, aes(x = cluster, y = valor, fill = cluster)) +
  geom_violin(trim = FALSE, alpha = 0.6) +
  geom_boxplot(width = 0.2, color = "black", alpha = 0.7) +
  facet_wrap(~variable, scales = "free_y") +
  theme_minimal(base_size = 12) +
  labs(title = "Distribución de contaminantes por clúster",
       x = "Clúster", y = "Valor escalado")

```

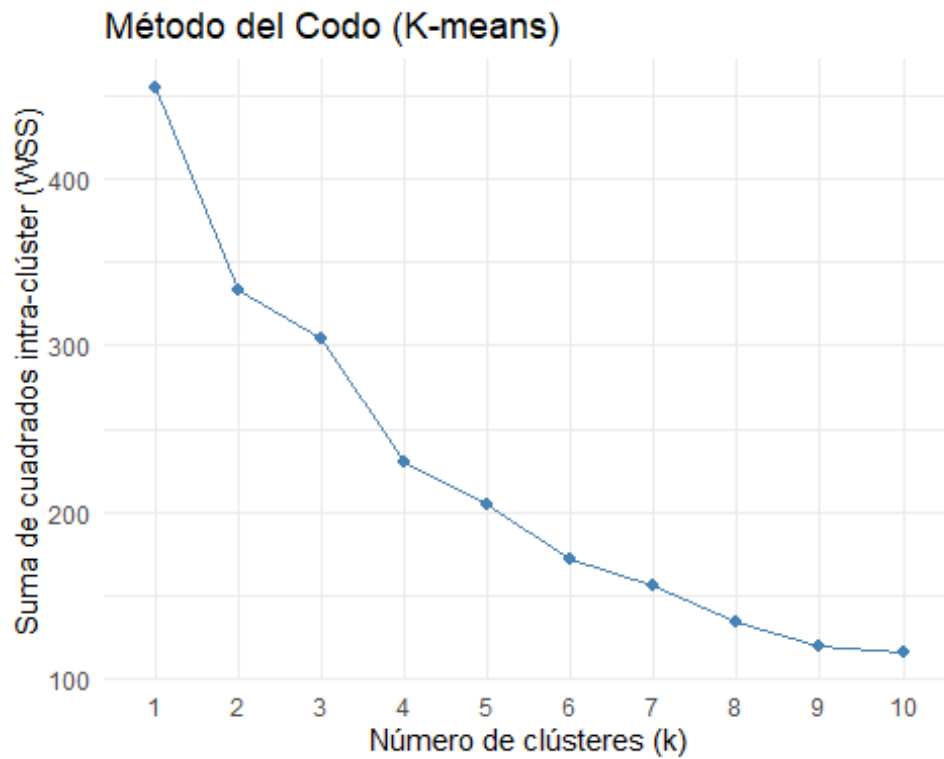

Distribución de contaminantes por clúster



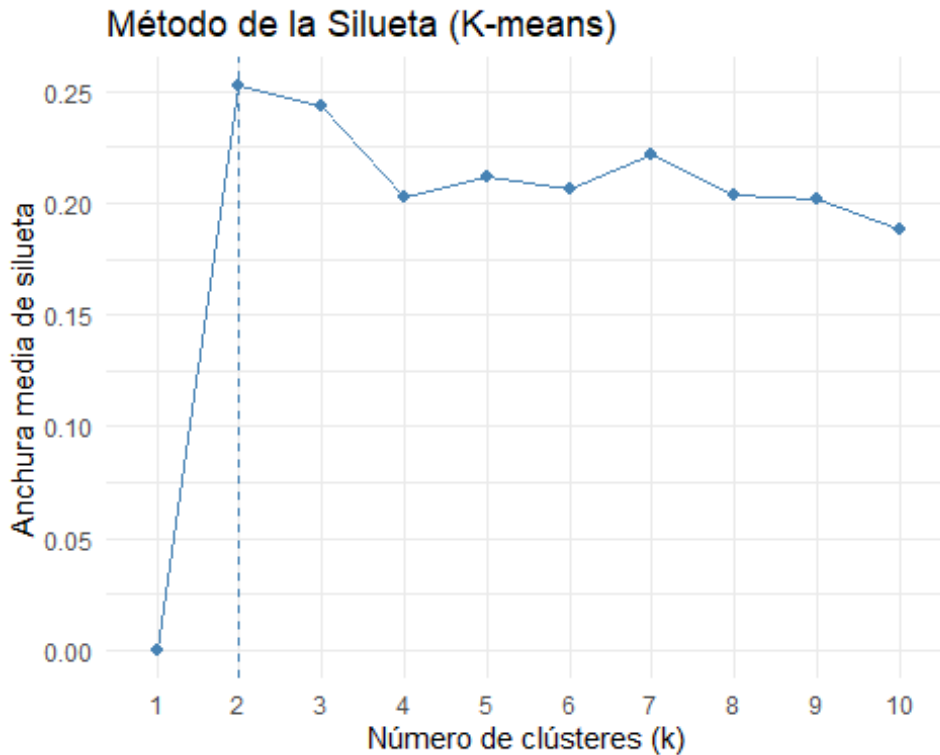
```
library(cluster)
library(factoextra)

# Supongamos que ya tienes tus datos escalados:
# X_scaled <- scale(tus_datos)

# 1) Método del Codo
fviz_nbclust(X_scaled, kmeans, method = "wss") +
  labs(title = "Método del Codo (K-means)",
       x = "Número de clústeres (k)",
       y = "Suma de cuadrados intra-clúster (WSS)") +
  theme_minimal()
```



```
# 2) Método de la Silueta
fviz_nbclust(X_scaled, kmeans, method = "silhouette") +
  labs(title = "Método de la Silueta (K-means)",
        x = "Número de clústeres (k)",
        y = "Anchura media de silueta") +
  theme_minimal()
```



```
suppressPackageStartupMessages({
  library(cluster)
  library(factoextra)
})

# 1) Preparar X_scaled de forma segura (usa tus datos originales en X)
# Si ya tienes X_scaled, comenta esta parte y deja tu objeto.
# X <- tus_datos |> as.data.frame()
stopifnot(exists("X_scaled") || exists("X"))
if (!exists("X_scaled")) {
  stopifnot(exists("X"))
  X <- as.data.frame(X)
  # imputación simple para evitar NAs
  X[] <- lapply(X, function(col) {
    if (all(is.na(col))) return(col)
    replace(col, is.na(col), median(col, na.rm = TRUE))
  })
  # eliminar columnas constantes (varianza 0)
  sd_ok <- sapply(X, function(z) sd(z, na.rm = TRUE) > 0)
  if (!all(sd_ok)) message("🔍 Quitando columnas constantes: ",
    paste(names(X)[!sd_ok], collapse=", "))
  X <- X[, sd_ok, drop = FALSE]
  X_scaled <- scale(X)
}

# Etiquetas seguras
```

```

if (is.null(rownames(X_scaled))) {
  rownames(X_scaled) <- paste0("obs_", seq_len(nrow(X_scaled)))
}

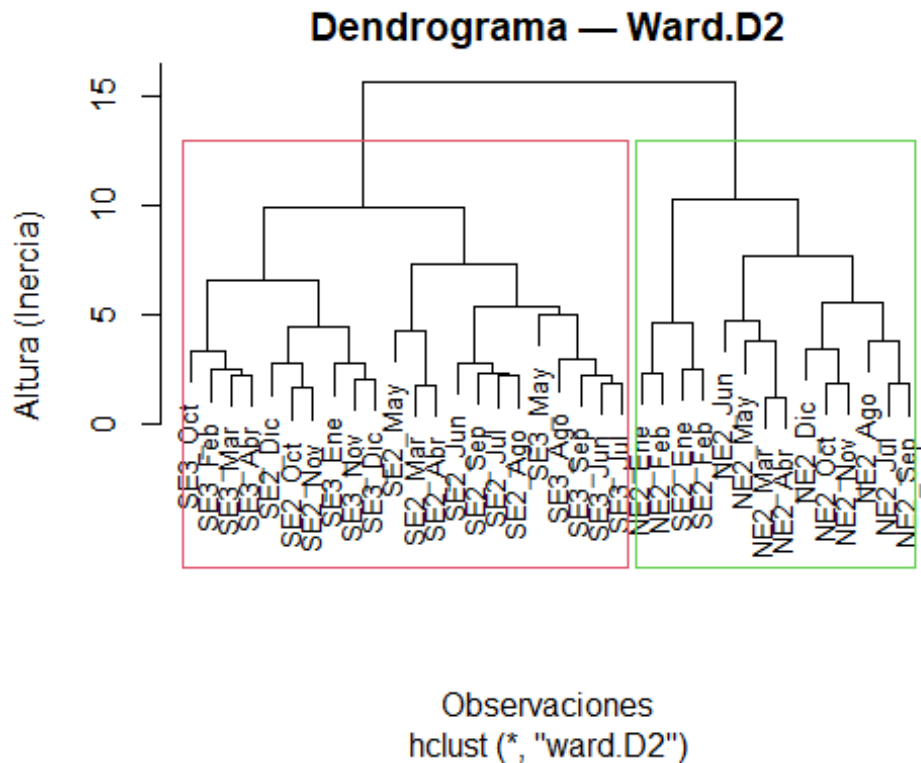
# Quitar filas con NA/Inf después del escalado
ok_rows <- apply(X_scaled, 1, function(r) all(is.finite(r)))
if (!all(ok_rows)) {
  message("👁 Quitando filas no finitas: ",
  paste(rownames(X_scaled)[!ok_rows], collapse = ", "))
  X_scaled <- X_scaled[ok_rows, , drop = FALSE]
}

stopifnot(nrow(X_scaled) >= 2) # necesita al menos 2 observaciones

# --- 2) Distancia y hclust (Ward.D2) ---
d <- dist(X_scaled, method = "euclidean")
if (any(!is.finite(d))) stop(" La matriz de distancias contiene NA/Inf.")
hc <- hclust(d, method = "ward.D2")
if (!inherits(hc, "hclust")) stop(" 'hc' no es un objeto 'hclust'.")

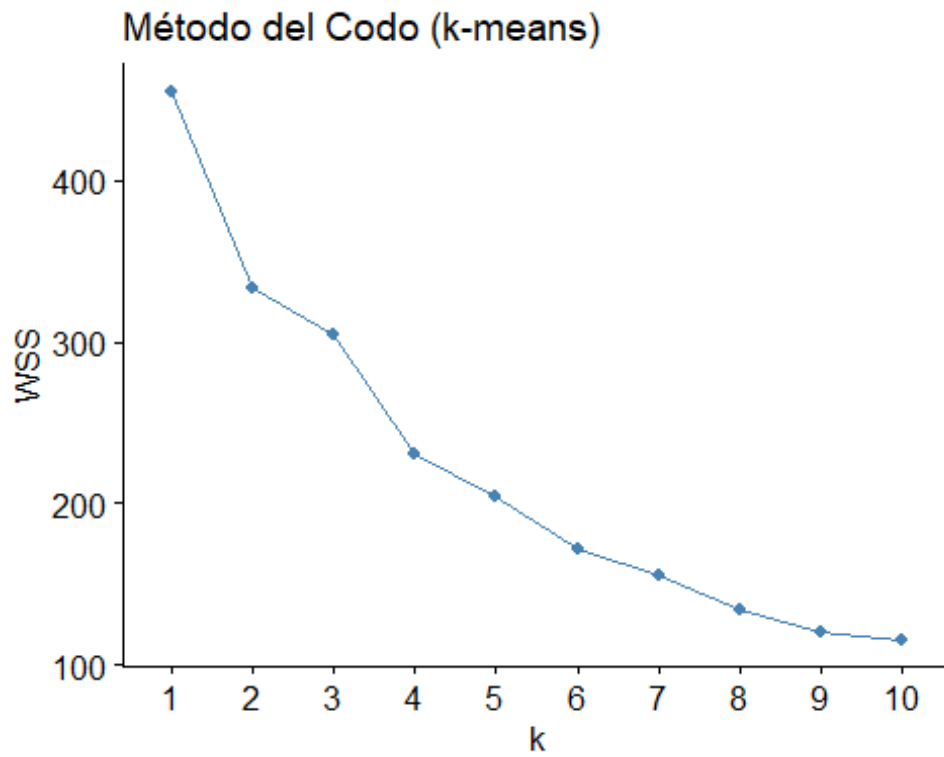
# 3) Dendrograma con rectángulos (k=2)
par(mar = c(5,4,2,1))
plot(hc,
  labels = rownames(X_scaled),
  main = "Dendrograma – Ward.D2",
  xlab = "Observaciones",
  ylab = "Altura (Inercia)",
  cex = 0.8)
rect.hclust(hc, k = 2, border = 2:3)

```



```
# (opcional) versión con factoextra, con rectángulos
# fviz_dend(hc, k = 2, rect = TRUE, rect_fill = TRUE,
#           main = "Dendrograma - Ward.D2 (k=2)")

# 4) Codo y Silueta (K-means) para referencia rápida
set.seed(123)
fviz_nbclust(X_scaled, kmeans, method = "wss") +
  labs(title = "Método del Codo (k-means)", x = "k", y = "WSS")
```



```
fviz_nbclust(X_scaled, kmeans, method = "silhouette") +  
  labs(title = "Método de la Silueta (k-means)", x = "k", y = "Silueta  
media")
```

